COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 13, 2026
NAME:	Alvarez, Joshua Caleb P Garcia, Vince Joseph R..	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 ALGORITHMIC STRATEGIES

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Tree traversals	Tree traversal algorithms recursively visit child subtrees until hitting a **nullptr** base case beyond a leaf node.
Brute Force	Linear search	Linear search is basically classifies as a brute force because it checks every item from index 0 to n-1 until it finds its target.
Backtracking	Depth-First search (DFS)	DFS explores as far as possible down one path or track and then comes back to try another
Greedy	Selection sort	In selection sorting the program do a repeatedly selects the smallest elements from the unsorted portion and places it in the correct or inorder position.
Divide-and-Conquer	Merge sort	In merge sort the program is the elements inside an array divides them into smaller parts, then sorts them and merges the results, which is placed in correct inorder position

Table 12-1. Algorithmic Strategies and Examples

Screenshot

```
4   int memo[160001];
5   int getMinSteps(int n)
6   {
7       if (n == 1) return 0;
8       if (memo[n] != -1) return memo[n];
9
10      int r = 1 + getMinSteps(n - 1);
11      if (n % 2 == 0)
12          r = std::min(r, 1 + getMinSteps(n / 2));
13      if (n % 3 == 0)
14          r = std::min(r, 1 + getMinSteps(n / 3));
15      memo[n] = r;
16
17      return r;
18  } <- #6-18 int getMinSteps(int n)
19  int main()
20  {
21      int n;
22      std::fill(memo, memo + 10001, -1);
23      std::cout << "Enter n: ";
24      std::cin >> n;
25
26      if (n < 1 || n > 10000) {
27      std::cout << "Enter a number from 1 to 10000.";
28          return 0;
29      }
30      std::cout << "Minimum Steps: " << getMinSteps(n);
31      return 0;
32  } <- #20-33 int main()
```

OUTPUT:

```
PS C:\Users\TIPQC\Desktop\AL
Enter n: 1
Minimum Steps: 0
PS C:\Users\TIPQC\Desktop\AL
Enter n: 2
Minimum Steps: 1
PS C:\Users\TIPQC\Desktop\AL
Enter n: 3
Minimum Steps: 1
```

```
PS C:\Users\TIPQC\Desktop\
Enter n: 10
Minimum Steps: 3
PS C:\Users\TIPQC\Desktop\
Minimum Steps: 2
PS C:\Users\TIPQC\Desktop\ALVAREZ_
Enter n: 100001
Enter a number from 1 to 10000.
PS C:\Users\TIPQC\Desktop\ALVAREZ
```

Analysis

In this code, I added some users' input and after they input which is n the program will run and output how many steps that the numbers count in the fibonacci sequences which it takes time for me to learn this memoization to find the minimum number of steps to reach 1. The program checks different operations, such as subtracting 1, dividing by 2 and dividing by 3 to find the shortest way, I also learned that saving the previous results helps the program to avoid repeating the same calculations. Based on the output, the program gives the correct minimum number of steps for the different inputs. Through this activity, I am slightly able to understand how recursion work together to solve a problem.

Table 12-2. Memoization Implementation

Screenshot	**Code:**
	`#include <iostream>`
	`#include <algorithm>`
	
	`int getMinSteps(int n){`
	
	`int* dp = new int[ n +1 ], i;`
	`dp[1] = 0; // trivial case`
	
	`for( i = 2 ; i <= n ; i ++ )`
	`{`
	`dp[i] = 1 + dp[i-1];`
	`if(i%2==0) dp[i] = std::min( dp[i] , 1+ dp[i/2] );`
	`if(i%3==0) dp[i] = std::min( dp[i] , 1+ dp[i/3] );`
	`}`
	`return dp[n];`
	`}`
	
	`int main(){`
	
	`int n = 10;`
	`std::cout<<"Minimum steps for n: "<<n<<": "<<getMinSteps(n)<<std::endl;`
	`}`
	Output:
	`Minimum steps for n: 10: 3`

Analysis	The bottom-up problem is solved because of this code by looping **2** to **n** and storing the best result for each number in **dp[i]** for every step it compares **i-1** with **i/2** or **i/3** choosing whichever route has the fewer moves. That way, it's much safer for large inputs than recursions so it will not crash the stack.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode

```
START

Algo CountPaths(matrix, row, col, cost, m, n)

   IF row >= m OR col >= n THEN
      RETURN 0
   ENDIF

   remainingCost = cost - matrix[row][col]

   IF remainingCost < 0 THEN
      RETURN 0
   ENDIF

   IF row == m - 1 AND col == n - 1 THEN
      IF remainingCost == 0 THEN
         RETURN 1
      ELSE
         RETURN 0
      ENDIF
   ENDIF

   rightPaths = CountPaths(matrix, row, col + 1, remainingCost, m, n)
   downPaths  = CountPaths(matrix, row + 1, col, remainingCost, m, n)

   RETURN rightPaths + downPaths

STOP


Algo Main()

   matrix = [[4, 7, 1, 6],
             [5, 7, 3, 9],
             [3, 2, 1, 2],
             [7, 1, 6, 3]]
   m = 4
   n = 4
   cost = 25

   RESULT = CountPaths(matrix, 0, 0, cost, m, n)
```

PRINT "Target Cost: ", cost
PRINT "Number of Paths: ", result

STOP

Working C++ Code

```cpp
//Alvarez and Garcia
#include <iostream>
#include <vector>

int countPaths(
    const std::vector<std::vector<int> >& matrix,
    int row, int col, int cost,
    int m, int n){
    if (row >= m || col >= n){
        return 0;
    }

    int remainingCost = cost - matrix[row][col];
    if (remainingCost < 0){
        return 0;
    }

    if (row == m - 1 && col == n - 1){
        return (remainingCost == 0) ? 1 : 0;
    }

    int rightPaths = countPaths(
        matrix, row, col + 1, remainingCost, m, n);
    int downPaths = countPaths(
        matrix, row + 1, col, remainingCost, m, n);

    return rightPaths + downPaths;
}
```

FIRST MATRIX(GIVEN IN EXAMPLE)

```
int main(){
    std::vector<std::vector<int> > matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int m = ;
    int n = 3;
    int cost = 25;

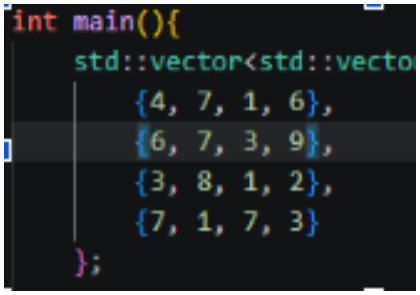
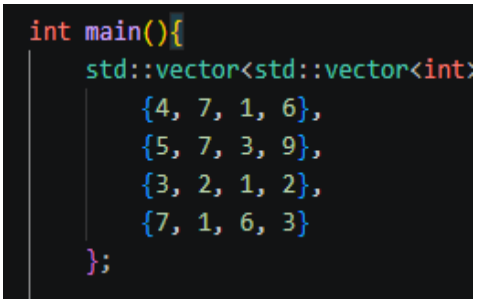
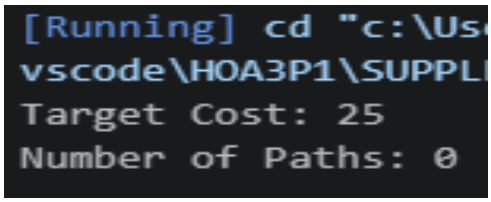
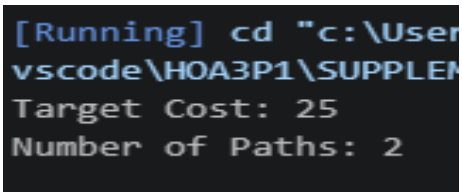
    int result = countPaths(matrix, 0, 0, cost, m, n);
    std::cout << "Target Cost: " << cost << '\n';
    std::cout << "Number of Paths: " << result << '\n';

    return 0;
}
```

SECOND MATRIX

```
30 int main(){
31     std::vector<std::vector<int> > matrix = {
32         {4, 7, 1, 6},
33         {5, 7, 3, 9},
34         {3, 2, 1, 2},
35         {7, 1, 6, 3}
36     };
37
38     int m = 4;
39     int n = 4;
40     int cost = 25;
41
42     int result = countPaths(matrix, 0, 0, cost, m, n);
43     std::cout << "Target Cost: " << cost << '\n';
44     std::cout << "Number of Paths: " << result << '\n';
45
46     return 0;
47 }
```

MATRIX TABLE

	FIRST MATRIX	SECOND MATRIX
CODE		
OUTPUT		
ANALYSIS	This is the matrix described in the activity. The program returns 0 because there is no path from **(0,0)** to **(3,3)** with a sum of **25**. The sums range from 21-37, and skip 25. The path shown in the sample activity **(4+7+1+3+1+2+3)** gives **21**, which is not equal to **25**. The program is performing correctly, and this data does not have a valid path at the given cost.	This matrix modifies the input of the values to match the grid that contains paths summing up to the target cost of **25**. The program outputs **2** because there are no exactly two valid paths from **(0, 0)** to **(3,3)** that total **25**. The recursive algorithm correctly identifies both valid paths while pruning invalid branches, confirming that the path-counting logic works as expected.

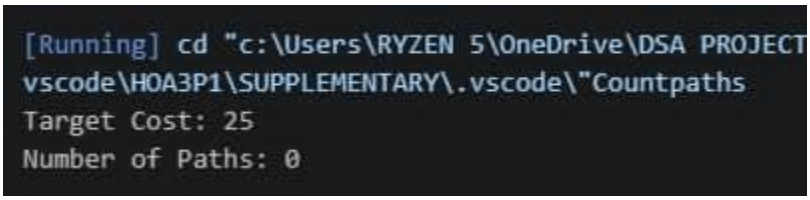
Analysis of Working Code

The program counts the **number of paths** from the **top-left cell** (0,0) to the **bottom-right cell** (M-1, N-1) of a matrix with a cost equal to a given target. Each cell in the matrix has a non-negative cost, and the cost of a path is the sum of every cell it steps on, including the starting cell and the destination cell. A path can move only one cell to the right, from (i, j) to (i, j+1), or one cell down, from (i, j) to (i+1, j), so it cannot move left, move up, or go diagonally. In main(), the program sets up a **4×4 matrix,** a target cost of 25, and the dimensions m and n. It then calls countPaths starting at (0,0) and prints the target cost and the number of matching paths.

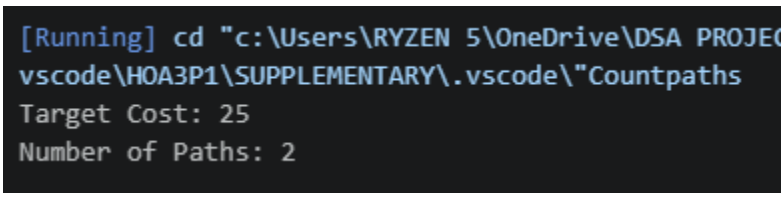
The program solves the problem **recursively**. It asks at each cell the same question, "how many ways can I finish from here with the cost I have left?", and answers it by splitting it into two smaller questions: how many paths continue to the right, and how many paths continue downward. It adds them together and returns it to the previous cell. To keep track of the cost, it subtracts the value of each cell from the target and passes it to the next calls. A path is valid only if it reaches the destination with exactly 0 cost remaining. Any path that finishes with something left over, or that goes over the target before the end, isn't counted. All the calls in the program process a cell that is closer to the destination, so the recursion always moves towards its base case and eventually stops.

Screenshots of Demonstration

FIRST MATRIX



SECOND MATRIX



CONCLUSION

ALVAREZ	In this activity, I learned how various algorithm strategies, such as recursion and dynamic programming work. I discovered that with recursion, a problem is broken down into smaller subproblems, and the bigger problem is solved by combining solutions to the subproblems. On the other hand, with dynamic programming, recursion is applied, but the results of the subproblems are stored to avoid recomputation. In the supplementary activity, I was able to apply recursion to count the number of paths in a matrix with a given cost. The matrix provided in the activity was a bit challenging, and initially, I got zero as the number of paths because none of the paths added up to 25, but later I realized that there were errors in the matrix and corrected three cells. Additionally, I faced a challenge in comparing our pseudocode to my code. However, I fixed this by comparing the code and pseudocode step by step. Finally, I encountered an error when compiling the program. I realized that the file was not in the right directory. These challenges taught me to first check my input (matrix or data) and setup (file directories) before concluding that the code is erroneous.
GARCIA	In this laboratory activity, I learned how recursive and dynamic programming approaches differs from solving path finding problems. Standard recursion breaks a problem down into smaller subproblems and solves them repeatedly while dynamic programming improves this process by storing subproblem results to avoid doing unnecessary work. For the supplementary activity I applied recursion to find and count all possible paths in a matrix that sum up to a target cost of 25. Working with the first matrix was difficult because running the program returned zero paths at first, as no valid path actually added up to 25. After checking the matrix values against the second matrix configuration, updating the numbers yielded two valid paths as expected. I also saw a problem matching the pseudocode logic to the working code syntax but it was resolved by breaking down the base cases step by step. Overall, the activity showed me the importance of verifying test data before assuming the code itself is broken.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.